

Day 1: Fundamentals of Image Processing

Comprehensive Study Notes & Theoretical Foundations

1. What an Image Is: The Digital Representation

A digital image is fundamentally a multidimensional array (or tensor) of numerical values. To a computer, an image is not a picture; it is a grid defined by three key dimensions: **Height** $\times$ **Width** $\times$ **Channels**.

Dimensions ($H \times W \times C$)

- **Height (H) & Width (W):** Represent the spatial resolution of the image (number of pixels).
- **Channels (C):** Represent the color depth. A grayscale image has $C = 1$, while a standard color image has $C = 3$ (Red, Green, Blue).

Data Types: uint8 vs float32

Property	uint8 (Unsigned 8-bit Integer)	float32 (32-bit Floating Point)
Range	0 to 255	0.0 to 1.0 (typically normalized)
Use Case	Storage, displaying images, standard I/O.	Mathematical computations, neural networks, avoiding overflow during filtering.
Memory	1 byte per pixel per channel. Highly efficient.	4 bytes per pixel per channel. Memory intensive.

Mathematical Implication: When performing operations like convolution, we often convert **uint8** to **float32**. If you add two **uint8** pixels (e.g., $200 + 100$), the result overflows (300 exceeds 255) leading to unexpected visual artifacts (wrap-around to 44). Floating point prevents this until we clip and cast back.

Memory Layout

How this data sits in RAM dictates execution speed, especially in Machine Learning contexts:

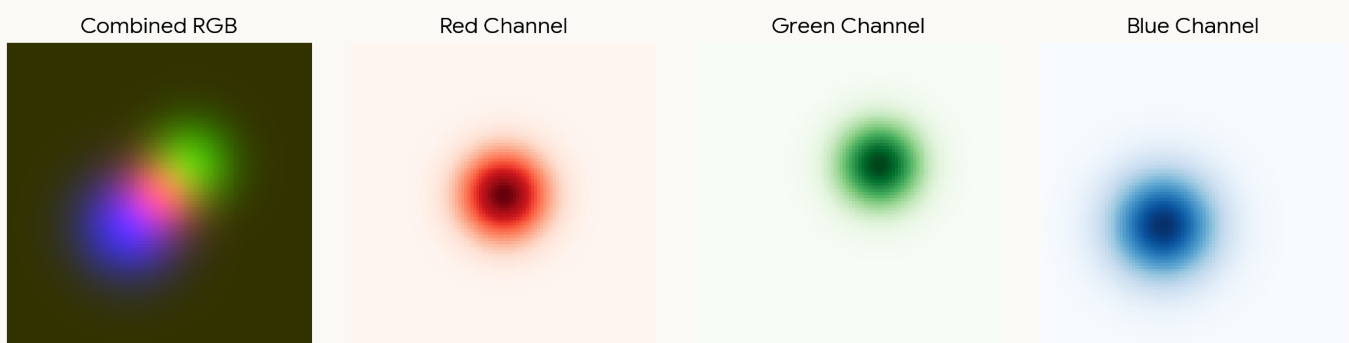
- **HWC (Interleaved):** Pixels are stored as [R,G,B, R,G,B...]. This is standard for OpenCV and displaying images because rendering engines draw pixel by pixel.
- **CHW (Planar):** Channels are stored sequentially [R,R,R... G,G,G... B,B,B...]. This is preferred by Deep Learning frameworks (PyTorch) because it allows for highly optimized vectorized matrix multiplications across spatial dimensions.

2. Colour Spaces: RGB, Grayscale, HSV

A color space is a mathematical model describing how colors can be represented as tuples of numbers.

RGB (Red, Green, Blue)

The additive default for hardware displays. While good for screens, RGB channels are highly correlated. If lighting changes, all three values (**R, G, B**) change simultaneously, making it terrible for object detection based on color.



Visualizing an image broken into its constituent R, G, and B matrices.

Grayscale

Reduces the image to structural intensity (luminance) by discarding color (chrominance). It drastically reduces computational load (from 3 channels to 1).

Conversion formula (Luma coding):

$$Y = 0.299R + 0.587G + 0.114B$$

HSV (Hue, Saturation, Value)

Designed to align with human perception.

- **Hue:** The actual color type (angle from 0 to 360).
- **Saturation:** The purity/intensity of the color (0 to 100%).
- **Value:** The brightness (0 to 100%).

Application Concept: Consider analyzing the prominent, vibrant yellows against the deep blues in a painting like Van Gogh's *The Starry Night*. In RGB, isolating "yellow" is complex because it requires strict combinations of R and G channels under varying lighting. In HSV, you simply isolate the **Hue** channel between specific angles (e.g., 20° to 40° for yellow), making color-masking invariant to shadows (Value) and fading (Saturation).

3. Geometric Transforms

Geometric transformations map pixels from a source coordinate (x, y) to a destination coordinate (x', y') .

- **Resize (Scaling):** Interpolates new pixel values. (Nearest-neighbor for speed, Bilinear/Bicubic for quality).
- **Crop:** Simply slicing the matrix: `img[y1:y2, x1:x2]`.

Affine Transformations

An affine transformation preserves collinearity (lines remain lines) and parallelism. It encompasses Translation, Rotation, Scaling, and Shearing. Mathematically, it is represented by a 2×3 transformation matrix M multiplied by the original coordinates (x, y) with an added homogeneous dimension (1).

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Where t_x, t_y handle translation, and a, b, c, d handle rotation, scale, and shear. For a pure rotation by angle θ :

$$a = \cos(\theta), b = -\sin(\theta), c = \sin(\theta), d = \cos(\theta)$$

4. Classical Convolution (Signal Processing)

Convolution is the mathematical foundation of image filtering and modern CNNs. It involves passing a small matrix, called a **Kernel** (or filter), over the image grid.

The Mathematics: Let I be the image and K be a $(2k+1) \times (2k+1)$ kernel. The discrete convolution operation is defined as:

$$(I * K)(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k I(x-i, y-j) K(i, j)$$

The Mechanism: The kernel "slides" over the image (stride). At each position, we perform an element-wise multiplication between the kernel and the overlapping image patch, sum the results, and place that single value in the output matrix.

*Note: True mathematical convolution requires flipping the kernel by 180 degrees first. If we do not flip it, the operation is technically called **Cross-Correlation**, though Deep Learning libraries simply call it convolution.*

5. Kernels in Action

Gaussian Blur (Smoothing)

Used to remove high-frequency noise. Unlike a simple box filter (which averages equally), a Gaussian kernel weights the center pixel highest, creating a smooth, natural-looking blur. It approximates the 2D Gaussian function: $G(x, y) = (1 / 2\pi\sigma^2) * e^{-(x^2+y^2) / 2\sigma^2}$

Sample 3x3 Gaussian approximation (multiplied by 1/16 to normalize):

$$\begin{vmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{vmatrix}$$

Sharpening

Sharpening enhances differences between adjacent pixels. It is often achieved by subtracting a blurred version of the image from the original, or by using a dedicated kernel that amplifies the center pixel relative to its neighbors:

$$\begin{vmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{vmatrix}$$

Edge Detection (Sobel, Laplacian, Canny)

Edges represent regions of rapid intensity change (high gradients).

1. Sobel Operator: Computes the first derivative (gradient) of the image. It uses two kernels to detect vertical (G_x) and horizontal (G_y) changes:

$$G_x = \begin{vmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{vmatrix} \quad G_y = \begin{vmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{vmatrix}$$

2. Laplacian Operator: Computes the second derivative of the image. It highlights regions of rapid change in any direction but is highly sensitive to noise.

3. Canny Edge Detector: Not just a kernel, but a multi-stage algorithm for robust edge detection:

- Noise Reduction:** Apply Gaussian Blur.
- Gradient Calculation:** Apply Sobel operators to find intensity gradients and their directions.
- Non-Maximum Suppression:** Thin the edges by keeping only local maxima along the gradient direction.
- Hysteresis Thresholding:** Use two thresholds (minVal, maxVal). Gradients above maxVal are strong edges. Gradients below minVal are discarded. Gradients in between are kept *only* if connected to a strong edge.

6. Histograms and Histogram Equalisation

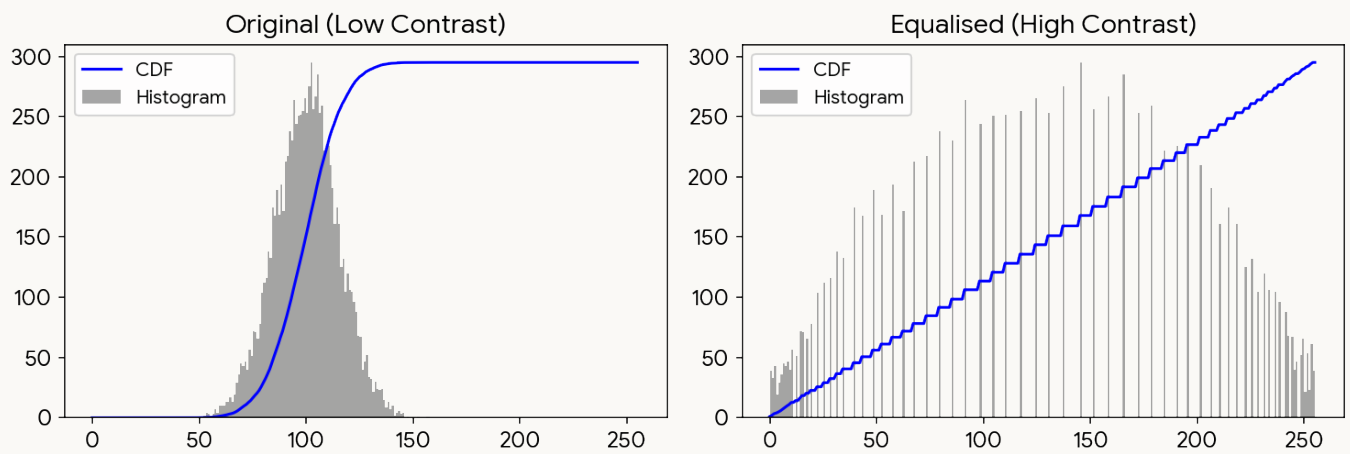
A histogram is a graphical representation of the distribution of pixel intensities in an image. For a grayscale image (0-255), the x-axis represents the 256 possible bins (pixel values), and the y-axis represents the frequency (number of pixels with that value).

Histogram Equalisation

Used to enhance global contrast. If an image is too dark, its histogram is clustered at the lower end. Equalisation mathematically spreads out the most frequent intensity values across the entire 0-255 range.

The Mathematics:

- Calculate the Probability Density Function (PDF) of the image intensities.
- Compute the Cumulative Distribution Function (CDF).
- Multiply the normalized CDF by the maximum intensity value (255) to map the old pixel values to new ones.



Notice how the low contrast image's narrow histogram (left) is stretched (right) by linearizing the Cumulative Distribution Function (CDF).

Summary Code Concept:

```
# OpenCV code for Equalisation
import cv2
img = cv2.imread('image.jpg', 0) # Load grayscale
equ = cv2.equalizeHist(img) # Enhances contrast mathematically
```